

Eleuthera

Guía del código

Cómo está organizado el proyecto, qué hace cada archivo
y dónde tocar para cambiar cada cosa.

Plataformas	Android 10+ · Windows · Linux · macOS
Tecnología	Kotlin 2.2 · Kotlin Multiplatform · Compose Multiplatform 1.8
Repositorio	`https://github.com/ErMichoss/Eleuthera`
Estado	rama `Dev-DesktopVersion`, octubre de 2026

Eleuthera (ἐλευθέρα) significa «libre» en griego antiguo.

Índice general

1 Visión general **1**

1.1 Qué es Eleuthera . . . 1

1.2 Los tres módulos . . . 1

1.3 Paquetes . . . 2

2 Árbol de carpetas **3**

2.1 Raíz del repositorio . . . 3

2.2 `shared/` . . . 3

2.3 `app/` . . . 4

2.4 `desktop/` . . . 4

3 Gradle, versiones y compilación **6**

3.1 Versiones . . . 6

3.2 Qué hace cada archivo de Gradle . . . 6

3.3 Comandos habituales . . . 7

4 Patrones que se repiten en todo el código **8**

4.1 *expect/actual*: lo que cada plataforma hace a su manera . . . 8

4.2 CompositionLocals: lo que la raíz de cada app «presta» a las pantallas . . . 8

4.3 Controllers comunes + envoltorios por plataforma . . . 9

4.4 Almacenes base abstractos . . . 9

4.5 Inyección de dependencias: Hilt frente a AppGraph . . . 10

5 Módulo `shared`: lo común **11**

5.1 Modelos (`domain/model`) . . . 11

5.2 Base de datos (`data/db`) . . . 11

5.3 Ajustes y almacenes (`data/settings`) . . . 12

5.4 Importación, YouTube y MusicBrainz . . . 12

5.5 Reproductor (`playback`) . . . 13

5.6 Pantallas (`ui`) . . . 13

5.7 `androidMain` . . . 14

6 Módulo `app`: Android **15**

6.1 Puntos de entrada y manifiesto . . . 15

6.2 Datos . . . 15

6.3 Reproducción 16
6.4 Interfaz 16

7 Escritorio: módulo `desktop` y `desktopMain` 18
7.1 La app (`desktop/`) 18
7.2 El motor (`shared/src/desktopMain`) 18
7.3 Instaladores 19

8 Cómo funciona cada cosa (flujos) 21
8.1 Buscar y descargar 21
8.2 Importar una lista de Spotify, Qobuz… 21
8.3 Biblioteca 21
8.4 Barra de progreso personalizada 22
8.5 Sesión de YouTube (cookies) 22
8.6 Aviso de versión nueva 22
8.7 MusicBrainz 22

9 Datos guardados 23
9.1 Base de datos `eleuthera.db` (Room, versión 4) 23
9.2 Ajustes (DataStore) 23
9.3 Archivos en disco 24

10 Publicar una versión (CI) 25

11 Quiero cambiar… 26

12 Restos, límites conocidos y pendientes 27
12.1 Archivos que sobran 27
12.2 Límites conocidos 27
12.3 Ideas pendientes 27

1

Visión general

1.1 Qué es Eleuthera

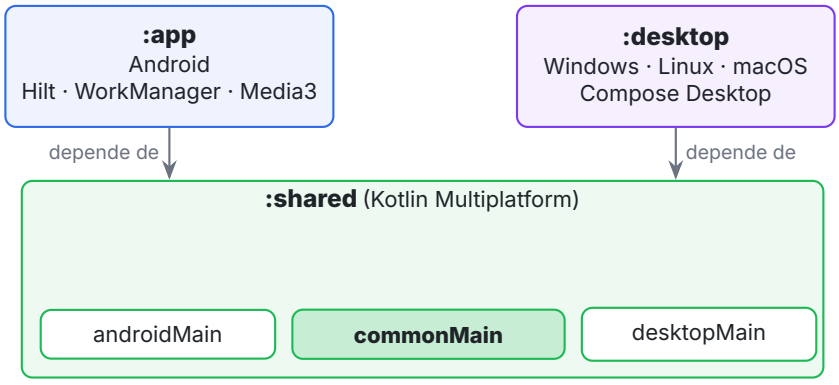
Eleuthera descarga música de YouTube, la organiza y la reproduce sin conexión. Además importa listas exportadas de Spotify, Qobuz, Apple Music y otros servicios: lee la lista de canciones, busca cada una en YouTube, elige la mejor versión y la descarga dentro de una playlist con el mismo nombre y orden.

Existe en dos «sabores» que comparten la mayor parte del código:

- ANDROID una app con navegación inferior, descargas en segundo plano (WorkManager), reproductor con notificación (Media3) y widget.
- ESCRITORIO una app para Windows, Linux y macOS con barra lateral, que descarga e instala sola yt-dlp, ffmpeg y Deno, y reproduce con un motor propio basado en ffmpeg.

1.2 Los tres módulos

El proyecto Gradle tiene tres módulos (ver `settings.gradle.kts`):



Módulo	Qué contiene
`shared/` COMÚN	Todo lo que es igual en las dos plataformas: modelos, base de datos, ajustes, lógica de las pantallas (*controllers*) y las propias pantallas Compose. Dentro tiene tres conjuntos de fuentes: `commonMain` (común de verdad), `androidMain` (las piezas comunes que en Android se hacen con APIs de Android) y `desktopMain` (lo mismo en escritorio, más todo el motor de escritorio: yt-dlp, ffmpeg, descargas, biblioteca…).

Módulo	Qué contiene
`app/` ANDROID	La app de Android: `Activity`, inyección con Hilt, descargas e importaciones con WorkManager, biblioteca con MediaStore, reproductor Media3, widget, inicio de sesión en YouTube. Sus pantallas son envoltorios finos sobre las de `shared`.
`desktop/` ESCRITORIO	El ejecutable de escritorio: `main()`, la ventana, la barra lateral, Ajustes de escritorio, el grafo de dependencias (`AppGraph`) y la configuración de los instaladores (.msi, .deb, .dmg).

La regla de oro

Si algo se ve o se comporta igual en el móvil y en el ordenador, va en `shared/src/commonMain`. Solo baja a `androidMain`/`desktopMain` (o a los módulos `app`/`desktop`) la parte que de verdad depende de la plataforma: abrir un archivo, leer una imagen, lanzar un proceso, pedir un permiso…

1.3 Paquetes

Todo el código está bajo el paquete `com.tuapp.eleuthera`, organizado por capas igual en los tres módulos:

Paquete	Qué va ahí
`domain.model`	Modelos puros, sin lógica: `Track`, `Playlist`, `RemoteTrack`, `ThemeConfig`, `ProgressSkin`…
`data.*`	Acceso a datos y servicios: `db` (Room), `settings` (DataStore, cookies, diseños), `youtube` (yt-dlp), `download`, `importer`, `library`, `playlist`, `update`, `musicbrainz`, `metadata`.
`playback`	Reproductor: la interfaz común `MusicPlayer` y sus dos implementaciones.
`platform`	Lo que abstrae la plataforma: `PlatformUri`, rutas y herramientas de escritorio.
`ui.*`	Pantallas Compose y su estado: `search`, `library`, `playlist(s)`, `player`, `settings`, `musicbrainz`, `common`, `theme`, `platform`.
`di`	Solo Android: el módulo de Hilt.
`widget`	Solo Android: el widget de la pantalla de inicio.
`desktop`	Solo escritorio: `main()`, ventana y pantallas propias.

Ojo con dos nombres parecidos

Las pantallas comunes de playlists están en el paquete `ui.playlist` (singular, en `shared`) y los envoltorios de Android en `ui.playlists` (plural, en `app`).

2

Árbol de carpetas

2.1 Raíz del repositorio

```
Eleuthera/
├── .github/workflows/release.yml   CI: APK + instaladores al subir una etiqueta vX.Y.Z
├── app/                            Módulo Android
├── shared/                         Módulo común (Kotlin Multiplatform)
├── desktop/                        Módulo de escritorio
├── docs/                           Esta documentación (LaTeX + PDF)
├── gradle/
│   ├── wrapper/                    Gradle 9.6.0 (gradlew lo descarga solo)
│   ├── gradle-daemon-jvm.properties  Gradle corre con el JDK 25 (lo descarga si falta)
│   └── libs.versions.toml          Resto de la plantilla: NO se usa (ver capítulo 11)
├── build.gradle.kts                Versiones de los plugins (Kotlin, AGP, Compose, KSP, Hilt)
├── settings.gradle.kts             Módulos :app, :shared, :desktop y repositorios
├── gradle.properties               Memoria de Gradle, caché de configuración, AndroidX
├── README.md                       Manual de usuario y guía rápida para desarrolladores
└── .gitignore                      build/, .gradle/, .kotlin/, local.properties, *.jks, cookies*
```

2.2 `shared/`

```
shared/
├── build.gradle.kts                Targets androidTarget + jvm("desktop"), Room/KSP, recursos
└── src/
    ├── commonMain/
    │   ├── composeResources/font/   inter, montserrat, nunito, playfair (.ttf variables)
    │   └── kotlin/com/tuapp/eleuthera/
    │       ├── domain/model/        Track, Playlist, RemoteTrack, ThemeConfig, ProgressSkin…
    │       ├── data/
    │       │   ├── db/              AppDatabase (Room v4), entidades y DAOs
    │       │   ├── importer/        ImportFileParser (CSV/TXT), TrackMatcher
    │       │   ├── musicbrainz/     MusicBrainzModel, MusicBrainzService
    │       │   ├── playlist/        PlaylistStore (base común)
    │       │   ├── settings/        PreferenceStores, Theme/Download/PlayerSettings,
    │       │   │                    ProgressSkinStore, LoadedSkin, CookieText
    │       │   └── youtube/         Resolved
    │       ├── platform/            PlatformUri (expect)
    │       ├── playback/            MusicPlayer (interfaz), PlayerState, QueueItem
    │       └── ui/
    │           ├── common/          NowPlaying, TrackArt, Selection, QueueMenu
    │           ├── library/         LibraryController, LibraryContent, TrackActions
    │           ├── musicbrainz/     MusicBrainzController, MusicBrainzContent
    │           ├── platform/        PlatformUi (expect + CompositionLocals)
    │           ├── player/          PlayerUi, MiniPlayer, NowPlayingScreen, SkinnedProgress
    │           ├── playlist/        PlaylistsController, PlaylistsContent, AddToPlaylistDialog
```

```
    │           ├── search/          SearchController (+SearchBackend), SearchContent
    │           ├── settings/        Apariencia, Descargas, Barra de progreso, piezas comunes
    │           └── theme/           AppTheme, AppComponents (AppCard/AppButton), SoftShadow
    ├── androidMain/kotlin/…         actual de Android: PlatformUri, PlatformUi, SoftShadow, BD
    └── desktopMain/kotlin/…         actual de escritorio + todo el motor de escritorio:
        ├── data/db/                 buildAppDatabase (SQLite incluido)
        ├── data/download/           DesktopDownloads (cola de descargas)
        ├── data/importer/           DesktopImports
        ├── data/library/            DesktopLibrary, Covers
        ├── data/playlist/           DesktopPlaylistStore
        ├── data/settings/           DesktopCookieStore, DesktopLibrarySettings,
        │                            DesktopProgressSkinStore
        ├── data/update/             DesktopUpdateChecker
        ├── data/youtube/            DesktopYtDlp
        ├── platform/                DesktopPaths, Tools (instalador de yt-dlp/ffmpeg/Deno),
        │                            PlatformUri.desktop (FileUri)
        ├── playback/                DesktopPlayerController, FfmpegAudioPlayer, Ffmpeg
        └── ui/                      PlatformUi.desktop, SoftShadow.desktop
```

2.3 app/

```
app/
├── build.gradle.kts               Versión desde VERSION_NAME, firma desde secretos, MusicBrainz
└── src/main/
    ├── AndroidManifest.xml        Permisos, Activities, PlaybackService, widget, FileProvider
    ├── res/                       Iconos (gato), layout del widget, temas, colores
    └── java/com/tuapp/eleuthera/
        ├── EleutheraApp.kt        Application (@HiltAndroidApp) + configuración de WorkManager
        ├── MainActivity.kt        Raíz: barra inferior, mini reproductor, CompositionLocals
        ├── di/AppModule.kt        Hilt: base de datos y PreferenceStores
        ├── data/
        │   ├── download/          DownloadScheduler, DownloadWorker, DownloadNotifier
        │   ├── importer/          ImportScheduler, ImportWorker, ImportViewModel, ImportPanel
        │   ├── library/           MediaLibrary (MediaStore), FolderLibrary (carpetas extra)
        │   ├── metadata/          MetaDataManager (en desarrollo, aún sin usar)
        │   ├── playlist/          PlaylistRepository (extiende PlaylistStore)
        │   ├── settings/          CookieStore, LibrarySettings, ProgressSkinRepository
        │   ├── update/            UpdateChecker (releases de GitHub)
        │   └── youtube/           YtDlpEngine (youtubedl-android)
        ├── playback/              PlaybackService (Media3), PlayerController
        ├── ui/                    Envoltorios de las pantallas comunes + lo propio de Android
        └── widget/                MusicWidgetProvider
```

2.4 desktop/

```
desktop/
├── build.gradle.kts               mainClass, versión, módulos de Java, instaladores
├── icons/                         icon.ico (Windows), icon.icns (macOS), icon.png (Linux)
└── src/main/
    ├── resources/icon.png         Icono de la ventana y de la barra de tareas
    └── kotlin/com/tuapp/eleuthera/desktop/
        ├── Main.kt                main(), ventana, barra lateral, mini reproductor, volumen
        ├── AppGraph.kt            Todas las piezas, creadas una vez (lo que Hilt en Android)
```

```
├── DesktopModels.kt       uiScope y los "ViewModels" de escritorio
├── SettingsScreen.kt      Ajustes de escritorio (7 páginas)
├── DownloadsScreen.kt     Cola de descargas y aviso de herramientas
├── ImportPanel.kt         Importar listas (CSV/TXT)
├── UpdatePrompt.kt        Aviso de versión nueva
├── RemoteImages.kt        Miniaturas de internet (lo que coil en Android)
└── FolderPicker.kt        Diálogos del sistema: carpeta, imagen, archivo
```

3

Gradle, versiones y compilación

3.1 Versiones

Las versiones de los plugins están en `build.gradle.kts` (raíz). Van ligadas entre sí: si se sube una, hay que revisar las demás.

Pieza	Versión y por qué
Kotlin	2.2.21. Es la primera rama que funciona con Java 25.
KSP	2.3.0 (genera el código de Room y Hilt).
Hilt	2.57.2 (solo Android).
Compose Multiplatform	1.8.2 (`org.jetbrains.compose`).
Android Gradle Plugin	8.7.3 (`compileSdk`/`targetSdk` 35, `minSdk` 29 = Android 10).
Gradle	9.6.0 (wrapper). El *daemon* usa el JDK 25 (`gradle/gradle-daemon-jvm.properties`); si no está, lo descarga con foojay.
Room	2.7.2 (KMP) con `sqlite-bundled` 2.5.2 en escritorio y `sqlite-framework` en Android.
DataStore	1.1.7 (`datastore-preferences-core` en común).
Coroutines / Serialization	1.9.0 / 1.7.3.
Media3	1.5.1 (ExoPlayer + MediaSession, Android).
youtubedl-android	0.18.1 (yt-dlp + FFmpeg + Python empaquetados para Android).
Coil	2.7.0 (imágenes en Android).
Reorderable	2.4.3 (arrastrar para reordenar, común).
Código generado	`jvmTarget` 17 en los tres módulos.

3.2 Qué hace cada archivo de Gradle

Archivo	Contenido
`settings.gradle.kts`	Repositorios (Google, Maven Central, Gradle Plugin Portal), plugin foojay para descargar JDKs y los tres módulos.
`build.gradle.kts`	Solo declara los plugins y sus versiones (`apply false`).
`gradle.properties`	`-Xmx2048m`, `org.gradle.configuration-cache=true`, AndroidX. La caché de configuración hace que cambiar una variable de entorno que lee un script (p. ej. `PACKAGING_JAVA_HOME`) obligue a reconfigurar: es normal.

Archivo	Contenido
`shared/build.gradle.kts`	Targets `androidTarget` y `jvm("desktop")`; dependencias de cada conjunto de fuentes; KSP de Room para los dos; recursos de Compose (clase `com.tuapp.eleuthera.shared.resources.Res`); `-Xexpect-actual-classes`.
`app/build.gradle.kts`	`versionName` desde `VERSION_NAME`, `versionCode` desde `GITHUB_RUN_NUMBER`; claves de MusicBrainz desde `local.properties` o variables de entorno (`BuildConfig`); firma de release solo si existe `SIGNING_STORE_FILE`; ABIs arm64-v8a, armeabi-v7a, x86_64.
`desktop/build.gradle.kts`	`mainClass`, versión (de la etiqueta, solo `X.Y.Z`), `-Deleuthera.version`, módulos de Java que van en el instalador y opciones de .msi/.deb/.dmg (icono, menú, `upgradeUuid` fijo, `bundleID`). `PACKAGING_JAVA_HOME` elige el JDK que se empaqueta.

3.3 Comandos habituales

```
./gradlew :desktop:run                                Abrir la app de escritorio
./gradlew :desktop:packageDistributionForCurrentOS    Instalador del sistema actual
                                                      -> desktop/build/compose/binaries/main/
./gradlew :app:assembleDebug                          APK de depuración
./gradlew :app:assembleRelease                        APK de release (firmado solo en CI)
./gradlew --stop                                      Parar el daemon (si algo se queda "pillado")
```

Empaquetar en tu ordenador

El Java que trae Android Studio (`jbr`) no incluye `jpackage`, así que el instalador falla con «jpackage.exe is missing». Instala un JDK 21 completo (por ejemplo `winget install EclipseAdoptium.Temurin.21.JDK`) y apunta a él: `$env:PACKAGING_JAVA_HOME = "C:\Program Files\Eclipse Adoptium\jdk-21…"`. Es lo mismo que hace GitHub Actions.

Si «no se ve ningún cambio»

`./gradlew --stop` y borrar `.kotlin/`, `desktop/build/` y `shared/build/` fuerza una compilación limpia.

4

Patrones que se repiten en todo el código

Entender estos cinco patrones es entender el 80 % del proyecto.

4.1 *expect/actual*: lo que cada plataforma hace a su manera

Kotlin Multiplatform permite declarar algo en `commonMain` con `expect` y darle una implementación `actual` en cada plataforma.

Declaración común	Android · Escritorio
`PlatformUri` `platform/PlatformUri.kt`	ANDROID `typealias` de `android.net.Uri` (`content://…`), así el código antiguo de Android no cambió. ESCRITORIO clase abstracta con una única implementación, `FileUri(URI)` (`file:///…`). El código común solo la guarda y la compara.
`PlatformBackHandler` `ui/platform/PlatformUi.kt`	ANDROID `BackHandler` de AndroidX. ESCRITORIO no hace nada: cada pantalla tiene su botón de volver.
`decodeImage(bytes)`	ANDROID `BitmapFactory`. ESCRITORIO Skia (`Image.makeFromEncoded`).
`drawSoftShadow` `ui/theme/SoftShadow.kt`	Sombra difuminada de color para el estilo neumórfico. ANDROID `Paint.setShadowLayer`. ESCRITORIO filtro de sombra de Skia (sigma ≈ radio/2).
`AppDatabaseConstructor`	Lo genera Room para cada plataforma; no se escribe a mano. Cada plataforma tiene además su `buildAppDatabase(…)`, que decide dónde está el archivo.

4.2 CompositionLocals: lo que la raíz de cada app «presta» a las pantallas

Las pantallas comunes no saben mostrar un aviso ni descargar una imagen. Lo piden a través de `CompositionLocal`s que pone la raíz de cada app (`MainActivity.kt` en Android, `Main.kt` en escritorio). Si faltan, no hacen nada en vez de fallar.

CompositionLocal	Para qué · quién lo pone
`LocalToast`	Aviso breve. ANDROID `Toast`. ESCRITORIO *snackbar*.
`LocalArtLoader`	Carátula de un archivo de música. ANDROID `androidArtLoader` (miniatura de MediaStore o la incrustada). ESCRITORIO `desktopArtLoader` → `Covers` (ffmpeg).
`LocalRemoteImage`	Imagen de internet (miniaturas de YouTube, carátulas de MusicBrainz). ANDROID coil. ESCRITORIO `desktopRemoteImage` (caché propia).
`LocalNowPlaying`	Qué canción suena, para resaltarla en las listas.

CompositionLocal	Para qué · quién lo pone
`LocalComponentStyle`	Estilo de tarjetas y botones elegido en Ajustes (plano, elevado, contorno, neumórfico). Lo pone `AppTheme`.

> **Cuidado**
> Una `Activity` nueva en Android no hereda los CompositionLocals de `MainActivity`: tiene que ponerlos ella (mira `MusicBrainzActivity`).

4.3 Controllers comunes + envoltorios por plataforma

La lógica de cada pantalla vive en una clase común normal (un *controller*) que recibe un `CoroutineScope`. En Android, un `ViewModel` de Hilt la crea con su `viewModelScope` y la expone como `val controller`; en escritorio la crea `AppGraph` con `uiScope`. Cuando una pantalla solo necesita unos pocos flujos, en vez de controller hay una *interfaz* que implementan el ViewModel de Android y una clase de escritorio.



Común (shared)	Android (app)	Escritorio
`SearchController` + `SearchBackend`	`SearchViewModel` (YtDlpEngine + DownloadScheduler)	`AppGraph.search` (DesktopYtDlp + DesktopDownloads)
`LibraryController`	`LibraryViewModel` (MediaLibrary)	`DesktopLibraryModel` (DesktopLibrary)
`PlaylistsController`	`PlaylistsViewModel` (PlaylistRepository)	`AppGraph.playlists` (DesktopPlaylistStore)
`ProgressSkinController`	`ProgressSkinViewModel` (ProgressSkinRepository)	`AppGraph.skins` (DesktopProgressSkinStore)
`MusicBrainzController`	`MusicBrainzViewModel` (+ OAuth)	`AppGraph.musicBrainz`
interfaz `PlayerUi`	`PlayerViewModel`	`DesktopPlayerUi`
interfaz `ThemeUi`	`ThemeViewModel`	`DesktopThemeUi`
interfaz `DownloadSettingsUi`	`DownloadSettingsViewModel`	`DesktopDownloadSettingsUi`
interfaz `MusicPlayer`	`PlayerController` (Media3)	`DesktopPlayerController` (ffmpeg)

4.4 Almacenes base abstractos

Cuando un almacén es igual en todo salvo una operación (leer una imagen, por ejemplo), la clase común es abstracta y cada plataforma solo implementa ese hueco:

Clase común	Lo que pone cada plataforma
`PlaylistStore` `data/playlist/PlaylistStore.kt`	`writeCover(image, dest, maxPx)`: guardar la portada elegida reducida. ANDROID `PlaylistRepository` (Bitmap; además importa las carpetas de playlists antiguas una vez). ESCRITORIO `DesktopPlaylistStore` (ffmpeg).
`ProgressSkinStore` `data/settings/ProgressSkinStore.kt`	`inspect(image)` (formato, peso, medidas) y `writePng(image, maxWidth, dest)`. ANDROID `ProgressSkinRepository` (ContentResolver + Bitmap). ESCRITORIO `DesktopProgressSkinStore` (Skia).

4.5 Inyección de dependencias: Hilt frente a AppGraph

- ANDROID Hilt. `di/AppModule.kt` solo crea la base de datos y los `PreferenceStores`; el resto de clases tienen `@Inject constructor`. Las clases comunes llevan `@Inject`/`@Singleton` de `javax.inject`, que Hilt usa en Android y que en escritorio no hacen nada.
- ESCRITORIO `desktop/.../AppGraph.kt`: un `object` con un `val … by lazy { … }` por cada pieza. Es el único sitio donde se «cablea» la app de escritorio. Para añadir una pieza nueva, se añade ahí.

5

Módulo `shared`: lo común

Rutas relativas a `shared/src/commonMain/kotlin/com/tuapp/eleuthera/`.

5.1 Modelos (`domain/model`)

Archivo	Qué hace
`Track.kt`	Una canción que ya está en el dispositivo: id, `PlatformUri`, título, artista, álbum, duración, carpeta… Es lo que se reproduce y se guarda en playlists.
`RemoteTrack.kt`	Un resultado de YouTube aún sin descargar (id de vídeo, título, canal, duración, URL, miniatura).
`Playlist.kt`	Playlist lista para pintar: datos, tamaño, duración y carátulas para el mosaico de la portada.
`ImportedTrack.kt`	Canción leída de un archivo exportado de Spotify, Qobuz…
`DownloadStatus.kt`	Estado de una descarga (*sealed*): en cola, progreso, terminada, fallida.
`AudioFormat.kt`	MP3 (recodificado, compatible) o M4A (AAC tal cual, sin pérdida).
`ThemeConfig.kt`	Todo lo personalizable: cuatro colores ARGB, fuente (`AppFont`), radio de las esquinas y estilo de componentes (`ComponentStyle`).
`ProgressSkin.kt`	Diseño de barra de progreso: rutas de las cuatro imágenes (`SkinSlot`: indicador, reproducida, pendiente, final), modo (`TrackMode`: Repetir, Estirar, Revelar, Perseguir), giro del indicador y colores lateral/central (Perseguir).

5.2 Base de datos (`data/db`)

Room multiplataforma. El archivo es `eleuthera.db`; en Android sigue en el mismo sitio de siempre, así que al actualizar se conservan las playlists.

Archivo	Qué hace
`AppDatabase.kt`	La base de datos (versión 4), sus tres DAOs y las migraciones 1→2→3→4. `MIGRATIONS` las usan las dos plataformas.
`PlaylistEntities.kt`	Tablas `playlists` y `playlist_tracks`.
`PlaylistDao.kt`	Consultas de playlists: observar, añadir sin repetir, `addToNamed` (crea la playlist si no existe, en una transacción), reordenar…

Archivo	Qué hace
DownloadEntities.kt	Tablas video_tracks (vídeo de YouTube → canción) y playlist_requests (peticiones pendientes de añadir a una playlist) y su DownloadDao.
FolderTrackEntities.kt	Tabla folder_tracks: etiquetas ya leídas de las canciones de carpetas añadidas, para no reabrir archivos que no han cambiado.

El esquema completo está en el capítulo 9.

5.3 Ajustes y almacenes (data/settings)

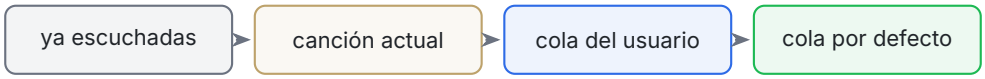
Archivo	Qué hace
PreferenceStores.kt	Abre los archivos DataStore en <baseDir>/datastore/<nombre>.preferences_pb (el mismo sitio que usaba Android) y garantiza una sola instancia por archivo.
ThemeRepository.kt	Tema (store theme).
DownloadSettings.kt	Formato de descarga (store downloads).
PlayerSettings.kt	Volumen de escritorio (store player).
ProgressSkinStore.kt	Base de los diseños de barra: skins.json + imágenes en <dir>/<id>/, validación (PNG/WebP, ≤1 MB, medidas por tipo de imagen), diseño activo ya cargado (active: Flow<LoadedSkin?>).
LoadedSkin.kt	Diseño activo con las imágenes ya decodificadas (se abren una vez, no en cada fotograma).
CookieText.kt	Convierte cookies JSON (Cookie-Editor…) a formato Netscape y las cuenta. Lo usa escritorio; Android aún tiene su propia conversión en CookieStore.

5.4 Importación, YouTube y MusicBrainz

Archivo	Qué hace
data/importer/ImportFileParser.kt	Lee CSV de Exportify, TuneMyMusic o Soundiiz (detecta columnas por su nombre) y texto plano «Artista - Título».
data/importer/TrackMatcher.kt	Elige el mejor resultado de YouTube: duración parecida, canal oficial ("Artista - Topic") y penaliza directos, covers, etc.
data/youtube/Resolved.kt	Resultado de resolver un enlace: vídeo suelto o playlist.
data/musicbrainz/MusicBrainzModel.kt	Respuestas de la API (kotlinx-serialization).
data/musicbrainz/MusicBrainzService.kt	Cliente HTTP (HttpURLConnection): buscar grabaciones y lanzamientos, detalles, OAuth. User-Agent identificando la app.
data/playlist/PlaylistStore.kt	Base común de las playlists (ver capítulo 4).

5.5 Reproductor (playback)

playback/MusicPlayer.kt define la interfaz MusicPlayer, PlayerState, QueueItem y RepeatMode. La cola funciona igual en las dos plataformas:



- Al tocar una canción, la *cola por defecto* es el resto de esa lista (o la lista barajada si el aleatorio está activo).
- «Reproducir a continuación» y «Añadir a la cola» van a la *cola del usuario*, que suena antes y se conserva al cambiar de lista.

5.6 Pantallas (ui)

Archivo	Qué hace
ui/platform/PlatformUi.kt	expects de interfaz y los CompositionLocals.
ui/theme/AppTheme.kt	Tema completo a partir de ThemeConfig: calcula superficies y textos con contraste; fuentes variables de composeResources.
ui/theme/AppComponents.kt	AppCard y AppButton con el estilo elegido; Modifier.neumorphic. Las pantallas usan estos en vez de Card/Button.
ui/common/NowPlaying.kt	Resaltado de la canción que suena y barras animadas.
ui/common/TrackArt.kt	Carátulas (con caché) y portada de playlist (mosaico 2×2).
ui/common/Selection.kt	Selección múltiple: barra de acciones y gestos de fila.
ui/common/QueueMenu.kt	Menú «a la cola / a una playlist» para listas enteras.
ui/library/*	Biblioteca: LibraryController (búsqueda sin tildes, pestañas canciones/artistas/álbumes/carpetas) y LibraryContent; TrackActions reúne las acciones sobre canciones que conecta cada raíz.
ui/search/*	Buscar: SearchController, interfaz SearchBackend y SearchContent (con un hueco header para el panel de importar).
ui/playlist/*	Cuadrícula y detalle de playlists, portada, reordenar y el diálogo «Añadir a playlist» (PlaylistTarget).
ui/player/*	PlayerUi, MiniPlayer (con fullControls en escritorio), NowPlayingScreen (cola integrada) y SkinnedProgress (barra con imágenes: SkinnedSeekBar y SkinnedTrack).
ui/settings/SettingsCommon.kt	Piezas de Ajustes: SettingsPageScaffold, SectionTitle, ScrollRow, SettingsHint y OptionChip (sustituye a FilterChip, que en escritorio dejaba un rectángulo claro al pasar el ratón).
ui/settings/SettingsUi.kt	Interfaces ThemeUi y DownloadSettingsUi; conversión HSV ↔ ARGB (no existe android.graphics.Color en escritorio).
ui/settings/AppearanceSettings.kt	Ajustes > Apariencia: temas rápidos, colores, selector libre, fuente, bordes, estilo y vista previa.
ui/settings/DownloadSettingsPage.kt	Ajustes > Descargas (formato).
ui/settings/ProgressSkin*.kt	Ajustes > Barra de progreso: controller y página (ImagePicker = selector de imágenes de la plataforma).

Archivo	Qué hace
ui/musicbrainz/*	Explorador de MusicBrainz: MusicBrainzController (estado de Compose) y MusicBrainzContent (búsqueda, resultados y ventana de detalles).

5.7 androidMain

Rutas relativas a shared/src/androidMain/kotlin/com/tuapp/eleuthera/.

Archivo	Qué hace
platform/PlatformUri.android.kt	actual typealias PlatformUri = android.net.Uri.
ui/platform/PlatformUi.android.kt	BackHandler y BitmapFactory.
ui/theme/SoftShadow.android.kt	Sombras con Paint.setShadowLayer.
data/db/AppDatabase.android.kt	buildAppDatabase(context) en databases/eleuthera.db.

6

Módulo $_{app}$: Android

Rutas relativas a `app/src/main/java/com/tuapp/eleuthera/`.

6.1 Puntos de entrada y manifiesto

Componente	Qué es
`EleutheraApp`	`Application` con `@HiltAndroidApp`; configura WorkManager para que los Workers reciban dependencias de Hilt.
`MainActivity`	La raíz: tema, CompositionLocals, barra inferior (Buscar, Biblioteca, Playlists, Ajustes), mini reproductor, reproductor completo, «Añadir a playlist», playlists rápidas y aviso de actualización.
`CookieLoginActivity`	Inicio de sesión en YouTube con un navegador interno; al guardar copia las cookies al `cookies.txt` de yt-dlp y las borra del navegador.
`MusicBrainzActivity`	Pantalla de MusicBrainz y vuelta del OAuth (`com.tuapp.eleuthera://oauth`).
`PlaybackService`	`MediaSessionService`: dueño del reproductor; Media3 genera la notificación y los controles de bloqueo, Bluetooth y auriculares.
`MusicWidgetProvider`	Widget de la pantalla de inicio (tema y diseño de barra).
Permisos	Internet, notificaciones, leer audio (`READ_MEDIA_AUDIO` / `READ_EXTERNAL_STORAGE` antiguo), servicios en primer plano (datos y reproducción), *wake lock*.

6.2 Datos

Archivo	Qué hace
`di/AppModule.kt`	Hilt: `AppDatabase` y `PreferenceStores(filesDir)`.
`data/youtube/YtDlpEngine.kt`	Única puerta a yt-dlp (librería youtubedl-android): inicializa y actualiza yt-dlp una vez por arranque, resuelve búsquedas/enlaces y descarga el audio (MP3 o M4A) a la caché.
`data/download/DownloadScheduler.kt`	Encola descargas en WorkManager, sin duplicados: si ya está descargada solo la añade a la playlist; si está en curso, apunta la playlist para cuando termine. Expone el estado por id de vídeo.
`data/download/DownloadWorker.kt`	Un trabajo por canción: descarga, publica en `Music/Eleuthera[/playlist]` y resuelve las peticiones de playlist. Máximo 4 a la vez (`DownloadLimiter`) y 2 reintentos.
`data/download/DownloadNotifier.kt`	Notificación de progreso con «Cancelar».

Archivo	Qué hace
`data/importer/ImportScheduler.kt`	Lanza importaciones (la lista va a un archivo) y recuerda las cerradas para que no reaparezcan.
`data/importer/ImportWorker.kt`	Busca cada canción, elige con `TrackMatcher` y la encola; apunta las no encontradas.
`data/importer/ImportViewModel.kt`, `ImportPanel.kt`	Confirmación (nombre de la playlist) y progreso de las importaciones, debajo del buscador.
`data/library/MediaLibrary.kt`	Biblioteca: `Music/Eleuthera` vía MediaStore (con etiquetas ID3) + carpetas extra; publicar y borrar archivos (con confirmación del sistema si Android la pide).
`data/library/FolderLibrary.kt`	Carpetas añadidas en Ajustes, recorridas con el permiso del selector (SAF), sin MediaStore. Lista barata + etiquetas en segundo plano guardadas en `folder_tracks`.
`data/playlist/PlaylistRepository.kt`	`PlaylistStore` de Android + importación única de carpetas de playlists antiguas.
`data/settings/CookieStore.kt`	`filesDir/cookies.txt`; importa Netscape/JSON y guarda la sesión de `CookieLoginActivity`; copia por proceso de yt-dlp.
`data/settings/LibrarySettings.kt`	Carpetas extra (URI + permiso persistente) y el botón flotante de playlists rápidas (activado y posición).
`data/settings/ProgressSkinRepository.kt`	`ProgressSkinStore` de Android.
`data/update/UpdateChecker.kt`	Última release de GitHub (API pública, cada 12 h como mucho; versiones omitidas).
`data/metadata/MetaDataManager.kt`	Metadatos de una canción (álbum, portada…) y detección de canciones sin carátula o sin datos. *En desarrollo: aún no lo usa nadie.*

6.3 Reproducción

Archivo	Qué hace
`playback/PlaybackService.kt`	El servicio de Media3 (sigue sonando sin interfaz; se apaga al quitar la app de recientes si no suena nada).
`playback/PlayerController.kt`	Implementa `MusicPlayer` hablando con el servicio por `MediaController`; la UI nunca toca ExoPlayer. Guarda acciones pedidas antes de conectar (p. ej. desde el widget) y redibuja el widget.

6.4 Interfaz

Archivo	Qué hace
`ui/search/*`	`SearchScreen` (pantalla común + `ImportPanel`) y `SearchViewModel`.
`ui/library/*`	`LibraryScreen` (permiso de lectura y confirmación de borrado alrededor de `LibraryContent`) y `LibraryViewModel`.

Archivo	Qué hace
`ui/playlists/*`	`PlaylistsScreen` (selector de fotos del sistema), `PlaylistsViewModel` y `QuickPlaylist.kt` (botón flotante arrastrable con todas las playlists).
`ui/player/PlayerViewModel.kt`	Implementa `PlayerUi` sobre `PlayerController`.
`ui/settings/SettingsScreen.kt`	Menú de Ajustes con resumen de cada categoría: Apariencia, Barra de progreso, Biblioteca, Descargas, Cuenta de YouTube, Acerca de, y el botón de MusicBrainz.
`ui/settings/*Page.kt`, `*ViewModel.kt`	Páginas propias de Android (Biblioteca, Cuenta, Acerca de) y los ViewModels que envuelven las comunes.
`ui/login/CookieLoginActivity.kt`	Inicio de sesión en YouTube.
`ui/musicbrainz/*`	`MusicBrainzActivity` y `MusicBrainzViewModel` (envoltorio).
`ui/update/UpdatePrompt.kt`	Aviso de versión nueva (+ `UpdateViewModel`).
`ui/common/AndroidArtLoader.kt`	Carátulas para `LocalArtLoader`.
`widget/MusicWidgetProvider.kt`	Widget redimensionable: carátula, controles y barra de progreso dibujada con el tema y el diseño activo (incluido Perseguir). Layout en `res/layout/widget\_music\_player.xml`.

7

Escritorio: módulo `desktop` y `desktopMain`

7.1 La app (`desktop/`)

Rutas relativas a `desktop/src/main/kotlin/com/tuapp/eleuthera/desktop/`.

Archivo	Qué hace
`Main.kt`	`main()`: aspecto nativo de los diálogos Swing, instala las herramientas (`AppGraph.prepareTools()`) y abre la ventana (1200×800, icono). `App()` es la raíz: barra lateral (Buscar, Biblioteca, Playlists, Ajustes), aviso de herramientas, contenido, mini reproductor con volumen, aviso de actualización, diálogo de playlists y reproductor completo encima.
`AppGraph.kt`	Todas las piezas, `by lazy`. Versión de la app desde `-Deleuthera.version`.
`DesktopModels.kt`	`uiScope` (hilo principal) y los equivalentes a ViewModels: `DesktopPlayerUi`, `DesktopLibraryModel`, `DesktopThemeUi`, `DesktopDownloadSettingsUi`, `desktopArtLoader`.
`SettingsScreen.kt`	Ajustes: Apariencia y Descargas (comunes), Barra de progreso (común), Biblioteca (carpetas del disco), Cuenta de YouTube (sesión del navegador o archivo), MusicBrainz (común) y Acerca de (versión, carpeta de datos, versiones de las herramientas, actualizar yt-dlp, buscar actualizaciones).
`DownloadsScreen.kt`	`DownloadQueuePanel` (a la derecha de Buscar) y `ToolsBanner` (instalando / error con «Reintentar»).
`ImportPanel.kt`	Importar CSV/TXT con confirmación y tarjetas de progreso.
`UpdatePrompt.kt`	Aviso de versión nueva (Descargar / Más tarde / Omitir).
`RemoteImages.kt`	Descarga y caché en memoria de imágenes de internet.
`FolderPicker.kt`	Diálogos del sistema: `choose()` (carpeta), `chooseImage()`, `chooseFile(título, extensiones)`.

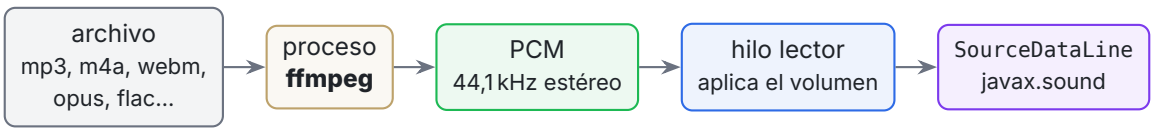
7.2 El motor (`shared/src/desktopMain`)

Rutas relativas a `shared/src/desktopMain/kotlin/com/tuapp/eleuthera/`.

Archivo	Qué hace
`platform/DesktopPaths.kt`	Carpeta de datos: `%APPDATA%\Eleuthera`, `~/Library/Application Support/Eleuthera` o `~/.local/share/eleuthera`. Música: `~/Music/Eleuthera`.

Archivo	Qué hace
platform/Tools.kt	Busca o descarga yt-dlp (releases de yt-dlp), ffmpeg/ffprobe (BtbN en Windows/Linux, martin-riedl en macOS) y Deno en <datos>/bin; actualiza yt-dlp en cada arranque; versiones para Acerca de.
platform/PlatformUri.desktop.kt	FileUri.
data/youtube/DesktopYtDlp.kt	Lanza yt-dlp: búsqueda/resolución (JSON), descarga con progreso, --ffmpeg-location, --js-runtimes deno:… (solo con nuestra copia), --cookies (copia por proceso) y exportBrowserCookies (--cookies-from-browser). Cancelar mata el proceso y sus hijos.
data/download/DesktopDownloads.kt	Cola de descargas: 4 a la vez, 2 reintentos, sin duplicados, subcarpeta y playlist con el nombre de la lista. *Vive en memoria.*
data/importer/DesktopImports.kt	Importaciones (TrackMatcher + cola de descargas). *Viven en memoria.*
data/library/DesktopLibrary.kt	Biblioteca: carpetas de DesktopLibrarySettings, relistado cada 30 s, etiquetas con ffprobe en lotes (4 hilos) guardadas en folder_tracks.
data/library/Covers.kt	Carátulas con ffmpeg (incrustada o cover.jpg / folder.jpg), 300 px, caché en disco (cache/covers) y en memoria.
data/playlist/DesktopPlaylistStore.kt	PlaylistStore con portadas vía ffmpeg.
data/settings/DesktopCookieStore.kt	<datos>/cookies.txt: importar archivo, traer del navegador (solo cookies de youtube.com/google.com y exige sesión iniciada).
data/settings/DesktopLibrarySettings.kt	Carpetas de música (store library).
data/settings/DesktopProgressSkinStore.kt	ProgressSkinStore con Skia.
data/update/DesktopUpdateChecker.kt	Release de GitHub con instalador para este sistema (.msi, .deb o el .dmg de su procesador). Si solo trae el APK, no avisa.
data/db/AppDatabase.desktop.kt	<datos>/eleuthera.db con SQLite incluido.
playback/Ffmpeg.kt	Localiza ffmpeg (bin/, PATH, rutas típicas) y probe() (etiquetas, duración, carátula).
playback/FfmpegAudioPlayer.kt	Reproduce cualquier formato (ver figura).
playback/DesktopPlayerController.kt	MusicPlayer de escritorio: la lista y la cola viven aquí; FfmpegAudioPlayer solo suena la canción actual.

Cómo suena una canción en escritorio



La posición la calcula la propia clase (lo enviado a la tarjeta menos lo que espera en su búfer); saltar a otro punto relanza ffmpeg desde ahí.

7.3 Instaladores

./gradlew :desktop:packageDistributionForCurrentOS genera el instalador del sistema en el que se ejecuta, con Java dentro (solo los módulos que usa la app: java.net.http, java.sql, java.naming,

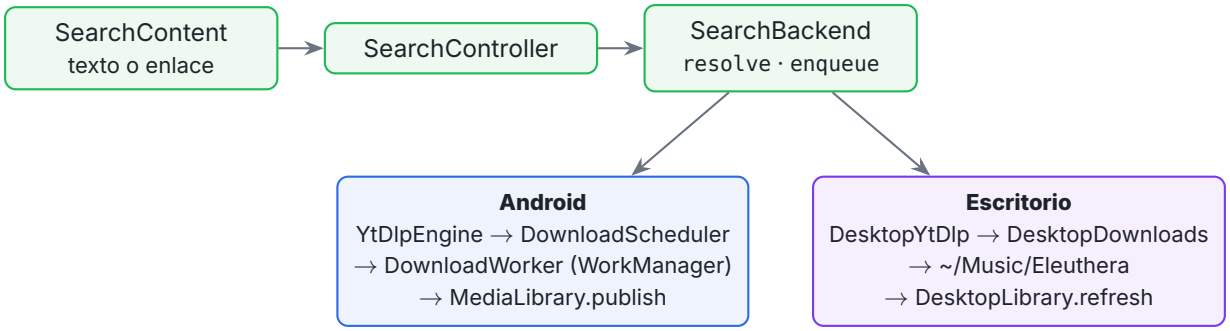
java.management, jdk.unsupported, jdk.charsets, jdk.localedata, jdk.accessibility y los que añade Compose).

Sistema	Detalles
Windows (.msi)	Por usuario (sin administrador), con elección de carpeta, menú y acceso directo. El upgradeUuid es fijo para instalar encima de la versión anterior.
Linux (.deb)	Paquete eleuthera, categoría AudioVideo, acceso directo.
macOS (.dmg)	com.tuapp.eleuthera, categoría Música. Sin firmar: la primera vez, clic derecho > Abrir (o xattr -dr com.apple.quarantine).

8

Cómo funciona cada cosa (flujos)

8.1 Buscar y descargar



1. El texto se convierte en `ytsearch20:…`; un enlace de vídeo o playlist se pasa tal cual. yt-dlp devuelve un JSON plano (`--flat-playlist`) que se traduce a `RemoteTrack`s.
2. Al descargar, si el vídeo ya está en `video_tracks` no se baja otra vez: solo se añade a la playlist. Si está en curso, se guarda una `playlist_request`.
3. yt-dlp extrae el audio (MP3 o M4A), incrusta metadatos y miniatura; el archivo va a `Eleuthera/` o `Eleuthera/<playlist>/` y se añade a la playlist en su posición.

8.2 Importar una lista de Spotify, Qobuz…

`ImportFileParser` lee el archivo y agrupa por playlist → el usuario confirma el nombre → por cada canción: buscar en YouTube, `TrackMatcher.best`, encolar la descarga dentro de la playlist (pausa corta entre búsquedas) → las no encontradas se pueden ver al final. Android: `ImportScheduler`/`ImportWorker` (sobrevive a cerrar la app). Escritorio: `DesktopImports` (en memoria).

8.3 Biblioteca

- ANDROID `MediaLibrary`: `Music/Eleuthera` vía MediaStore + carpetas extra con `FolderLibrary` (SAF).
- ESCRITORIO `DesktopLibrary`: `~/Music/Eleuthera` + carpetas añadidas, recorridas directamente.
- En ambas: los archivos nuevos aparecen al momento con el nombre del archivo («Artista - Título [id]») y sus etiquetas se leen en segundo plano y se guardan en `folder_tracks`, para no volver a abrir los que no cambian.
- `LibraryController` recibe la lista y hace búsqueda, pestañas y grupos.

8.4 Barra de progreso personalizada

Ajustes > Barra de progreso → `ProgressSkinController` → `ProgressSkinStore` valida la imagen (`inspect`), la reduce (`writePng`: 256 px las cuadradas, 2048 px las barras) y la guarda en `progress_skins/<id>/`. El diseño activo se expone ya decodificado (`LoadedSkin`) y lo dibujan `SkinnedSeekBar` (reproductor), `SkinnedTrack` (mini reproductor) y, en Android, el widget.

8.5 Sesión de YouTube (cookies)

yt-dlp necesita cookies para algunos vídeos. Se guardan en formato Netscape en la carpeta privada de la app y cada proceso de yt-dlp recibe *su propia copia* (yt-dlp reescribe el archivo y hay varias descargas a la vez).

- ANDROID importar archivo o iniciar sesión dentro de la app (`CookieLoginActivity`).
- ESCRITORIO importar archivo o «Traer sesión» del navegador (Firefox, Chrome, Edge, Brave, Opera, Vivaldi, Safari, Chromium): yt-dlp lee las cookies una vez y se guardan solo las de YouTube/Google, exigiendo que haya sesión iniciada.

8.6 Aviso de versión nueva

Al abrir (como mucho cada 12 h) se consulta `api.github.com/repos/ErMichoss/Eleuthera/releases/latest`. Android busca el APK; escritorio, el instalador de su sistema (y en macOS el `.dmg` de su procesador: `-mac-arm64` o `-mac-x64`). Se puede omitir una versión. La comprobación manual está en Acerca de.

8.7 MusicBrainz

`MusicBrainzController` busca canciones (*recordings*) o álbumes (*releases*) con consultas Lucene, abre los detalles respetando el límite de MusicBrainz (pausa de 1,2 s) y puede cargar los detalles de todas las pistas de un álbum. Carátulas de `coverartarchive.org`. El inicio de sesión (OAuth) solo está en Android: la redirección registrada es `com.tuapp.eleuthera://oauth`, que el escritorio no puede recibir.

9

Datos guardados

9.1 Base de datos `eleuthera.db` (Room, versión 4)

Tabla	Columnas y uso
`playlists`	`id` (PK autogenerada), `name`, `createdAt`, `coverPath` (portada personalizada, v2).
`playlist_tracks`	`playlistId`, `trackId` (id de la canción), `position`: orden de cada canción dentro de cada playlist.
`video_tracks`	`videoId` (PK) → `trackId`: qué archivo corresponde a cada vídeo de YouTube (v3, evita descargar dos veces).
`playlist_requests`	(`videoId`, `playlistName`) PK, `position`: «añade este vídeo a esta playlist cuando exista el archivo» (v3).
`folder_tracks`	`documentId` (PK), `uri`, `folder`, `lastModified`, `size`, `title`, `artist`, `album`, `durationMs`, `tagged`: etiquetas ya leídas de las carpetas añadidas (v4).

Cambiar el esquema

Si cambias una entidad: sube `version` en `AppDatabase`, escribe la `Migration` correspondiente y añádela a `MIGRATIONS`. Sin migración, los usuarios perderían sus playlists al actualizar.

9.2 Ajustes (DataStore)

Archivos `datastore/<nombre>.preferences_pb` dentro de la carpeta privada de la app.

Store	Claves
`theme` COMÚN	`primary`, `secondary`, `background`, `accent`, `font`, `radius`, `style`.
`downloads` COMÚN	`format`.
`player` COMÚN	`volume` (escritorio).
`library` ANDROID	`folder_trees` (carpetas con permiso), `extra_folders` (formato antiguo), `quick_playlists`, `quick_button_x/_y`.
`library` ESCRITORIO	`folders`.
`updates` ESCRITORIO	`last_check`, `skipped_version`.

En Android además hay *SharedPreferences* antiguas: `updates` (`last_check`, `skipped_version`), `imports` (`dismissed`) y `playlists` (importación única de carpetas).

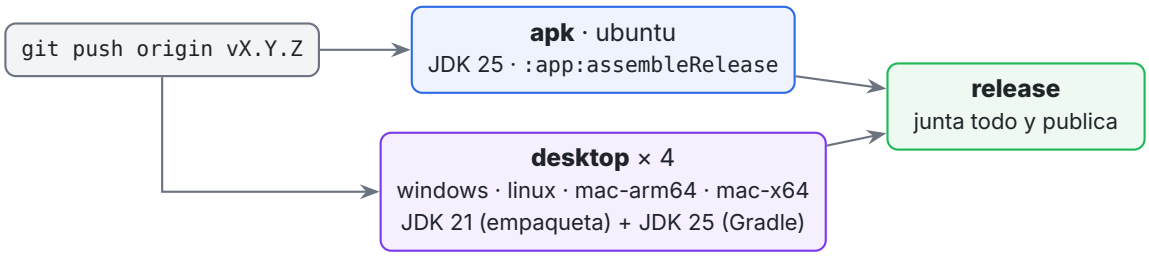
9.3 Archivos en disco

Dónde	Qué hay
ANDROID `filesDir/`	`datastore/`, `cookies.txt`, `covers/` (portadas de playlists), `progress_skins/` (`skins.json` + imágenes), `imports/` (no encontradas).
ANDROID `cacheDir/`	`dl/`, `resolve/` (temporales de yt-dlp), `imports/` (listas en curso).
ANDROID almacenamiento compartido	`Music/Eleuthera[/playlist]/`: la música.
ESCRITORIO carpeta de datos	`bin/` (yt-dlp, ffmpeg, ffprobe, deno), `datastore/`, `eleuthera.db`, `cookies.txt`, `covers/`, `progress_skins/`, `cache/covers/`, `cache/dl/`, `cache/tools/`.
ESCRITORIO `~/Music/Eleuthera/`	La música descargada.

10

Publicar una versión (CI)

`.github/workflows/release.yml` se lanza al subir una etiqueta `v*` (y a mano desde Actions > Release > Run workflow, para probar sin publicar).



Secreto de GitHub	Para qué
`KEYSTORE_BASE64`	Almacén de claves de firma en Base64.
`SIGNING_STORE_PASSWORD`, `SIGNING_KEY_ALIAS`, `SIGNING_KEY_PASSWORD`	Firma del APK.
`MUSICBRAINZ_CLIENT_ID`, `MUSICBRAINZ_CLIENT_SECRET`	Inicio de sesión en MusicBrainz (en local van en `local.properties`).

Archivos publicados: `Eleuthera-vX.Y.Z.apk`, `…-windows-x64.msi`, `…-linux-amd64.deb`, `…-mac-arm64.dmg` y `…-mac-x64.dmg`. El `versionCode` de Android es el número de ejecución, que siempre crece.

Nunca se sube al repositorio

El almacén de claves (`*.jks`) y sus contraseñas, `local.properties` (claves de MusicBrainz y ruta del SDK), cualquier `cookies*.txt`/`cookies*.json` y la carpeta `.idea/`. El `.gitignore` ya los excluye, pero revisa cada commit. Para las cookies, mejor una cuenta de YouTube secundaria.

11

Quiero cambiar…

Quiero…	Toca aquí
Una pantalla que se ve en las dos apps	`shared/.../ui/<pantalla>/*Content.kt` (y su `*Controller.kt`).
Algo solo del móvil (permisos, notificación, widget)	`app/`.
Algo solo del ordenador (ventana, barra lateral, Ajustes de escritorio)	`desktop/src/main/kotlin/.../desktop/`.
Cómo se descarga en escritorio	`DesktopYtDlp` y `DesktopDownloads` (`desktopMain`).
Cómo se descarga en Android	`YtDlpEngine`, `DownloadScheduler`, `DownloadWorker`.
Las opciones de yt-dlp (formato, metadatos)	`downloadAudio` en `YtDlpEngine` *y* en `DesktopYtDlp` (mantenerlas iguales).
Cómo se elige la canción al importar	`shared/.../data/importer/TrackMatcher.kt`.
Qué formatos de lista se aceptan	`ImportFileParser.kt`.
Colores, fuentes, temas rápidos	`ui/theme/AppTheme.kt`, `ui/settings/AppearanceSettings.kt` (`presets`).
Una fuente nueva	`shared/src/commonMain/composeResources/font/` + `AppFont` + `AppTheme`.
El aspecto de tarjetas y botones	`ui/theme/AppComponents.kt`.
Un modo nuevo de barra de progreso	`TrackMode` (`ProgressSkin.kt`), `SkinnedProgress.kt`, `ProgressSkinSection.kt` y el widget.
La cola de reproducción	`PlayerController` (Android) y `DesktopPlayerController` (escritorio): las dos siguen `MusicPlayer`.
Una tabla o columna de la base de datos	`data/db/` + migración (capítulo 9).
Un ajuste nuevo	El repositorio de su store en `data/settings/` y su página.
Una dependencia para las dos apps	`shared/build.gradle.kts` (`commonMain`).
Una pieza nueva en escritorio	Créala y añádela a `AppGraph`.
Una pieza nueva en Android	`@Inject constructor` (o un `@Provides` en `AppModule`).
MusicBrainz	`shared/.../data/musicbrainz/` y `shared/.../ui/musicbrainz/` (*no* en `app/`).
Los instaladores	`desktop/build.gradle.kts` y `desktop/icons/`.
La publicación	`.github/workflows/release.yml`.

12

Restos, límites conocidos y pendientes

12.1 Archivos que sobran

- `gradle/libs.versions.toml`: lo dejó la plantilla de Android Studio; ningún script lo usa (las versiones están en los `build.gradle.kts`). Se puede borrar.
- `app/src/main/java/com/tuapp/eleuthera/ui/theme/Color.kt`, `Theme.kt`, `Type.kt`: tema de la plantilla, sustituido por `AppTheme` de `shared`.
- `ExampleUnitTest` y `ExampleInstrumentedTest`: pruebas de ejemplo.

12.2 Límites conocidos

- ESCRITORIO La cola de descargas y las importaciones viven en memoria: si se cierra la app a medias, se pierden (en Android sobreviven gracias a WorkManager).
- ESCRITORIO MusicBrainz sin inicio de sesión (haría falta registrar otra redirección).
- ESCRITORIO La versión de macOS no está firmada por Apple.
- ANDROID `CookieStore` tiene su propia conversión JSON→Netscape; podría usar `CookieText` (común).
- ANDROID `MetaDataManager` está a medias y aún no se usa.

12.3 Ideas pendientes

Atajos de teclado y teclas multimedia en escritorio, icono en la bandeja del sistema para seguir sonando con la ventana cerrada, arrastrar un CSV o un enlace a la ventana, y cola de descargas persistente en escritorio.



Mantén este documento al día: si mueves o creas archivos, actualiza el capítulo correspondiente y vuelve a compilar `docs/estructura.tex`.